FastAPI Reference

Encoders - `jsonable_encoder`

func fastapi.encoders.jsonable_encoder

```
jsonable_encoder(
    obj,
    include=None,
    exclude=None,
    by_alias=True,
    exclude_unset=False,
    exclude_defaults=False,
    exclude_none=False,
    custom_encoder=None,
    sqlalchemy_safe=True,
)
```

Convert any object to something that can be encoded in JSON.

This is used internally by FastAPI to make sure anything you return can be encoded as JSON before it is sent to the client.

You can also use it yourself, for example to convert objects before saving them in a database that supports only JSON.

Read more about it in the FastAPI docs for JSON Compatible Encoder.

PARAMETER	DESCRIPTION
`obj`	The input object to convert to JSON. **TYPE:** `Any`
`include`	Pydantic's `include` parameter, passed to Pydantic models to set the fields to include. **TYPE:** `IncEx | None` **DEFAULT:** `None`
`exclude`	Pydantic's `exclude` parameter, passed to Pydantic models to set the fields to exclude. **TYPE:** `IncEx | None` **DEFAULT:** `None`
`by_alias`	Pydantic's `by_alias` parameter, passed to Pydantic models to define if the output should use the alias names (when provided) or the Python attribute names. In an API, if you set an alias, it's probably because you want to use it in the result, so you probably want to leave this set to `True`. **TYPE:** `bool` **DEFAULT:** `True`
`exclude_unset`	Pydantic's `exclude_unset` parameter, passed to Pydantic models to define if it should exclude from the output the fields that were not explicitly set (and that only had their default values). **TYPE:** `bool` **DEFAULT:** `False`
`exclude_defaults`	Pydantic's `exclude_defaults` parameter, passed to Pydantic models to define if it shou exclude from the output the fields that had the same default value, even when they were

Ask AI

PARAMETER	DESCRIPTION
	explicitly set. **TYPE:** `bool` **DEFAULT:** `False`
`exclude_none`	Pydantic's `exclude_none` parameter, passed to Pydantic models to define if it should exclude from the output any fields that have a `None` value. **TYPE:** `bool` **DEFAULT:** `False`
`custom_encoder`	Pydantic's `custom_encoder` parameter, passed to Pydantic models to define a custom encoder. **TYPE:** `dict[Any, Callable[[Any], Any]] | None` **DEFAULT:** `None`
`sqlalchemy_safe`	Exclude from the output any fields that start with the name `_sa`. This is mainly a hack for compatibility with SQLAlchemy objects, they store internal SQLAlchemy-specific state in attributes named with `_sa`, and those objects can't (and shouldn't be) serialized to JSON. **TYPE:** `bool` **DEFAULT:** `True`

Source code in `fastapi/encoders.py`

```python
def jsonable_encoder(
    obj: Annotated[
        Any,
        Doc(
            """
            The input object to convert to JSON.
            """
        ),
    ],
    include: Annotated[
        IncEx | None,
        Doc(
            """
            Pydantic's `include` parameter, passed to Pydantic models to set the
            fields to include.
            """
        ),
    ] = None,
    exclude: Annotated[
        IncEx | None,
        Doc(
            """
            Pydantic's `exclude` parameter, passed to Pydantic models to set the
            fields to exclude.
            """
        ),
    ] = None,
    by_alias: Annotated[
        bool,
        Doc(
            """
            Pydantic's `by_alias` parameter, passed to Pydantic models to define if
            the output should use the alias names (when provided) or the Python
            attribute names. In an API, if you set an alias, it's probably because you
            want to use it in the result, so you probably want to leave this set to
            `True`.
            """
        ),
    ] = True,
    exclude_unset: Annotated[
        bool,
        Doc(
            """
            Pydantic's `exclude_unset` parameter, passed to Pydantic models to define
            if it should exclude from the output the fields that were not explicitly
            set (and that only had their default values).
            """
        ),
    ] = False,
    exclude_defaults: Annotated[
        bool,
        Doc(
            """
            Pydantic's `exclude_defaults` parameter, passed to Pydantic models to define
            if it should exclude from the output the fields that had the same default
            value, even when they were explicitly set.
            """
        ),
    ] = False,
    exclude_none: Annotated[
        bool,
        Doc(
            """
            Pydantic's `exclude_none` parameter, passed to Pydantic models to define
            if it should exclude from the output any fields that have a `None` value.
            """
        ),
    ] = False,
    custom_encoder: Annotated[
        dict[Any, Callable[[Any], Any]] | None,
        Doc(
            """

```

```
            Pydantic's `custom_encoder` parameter, passed to Pydantic models to define
            a custom encoder.
            """
        ),
    ] = None,
    sqlalchemy_safe: Annotated[
        bool,
        Doc(
            """
            Exclude from the output any fields that start with the name `_sa`.

            This is mainly a hack for compatibility with SQLAlchemy objects, they
            store internal SQLAlchemy-specific state in attributes named with `_sa`,
            and those objects can't (and shouldn't be) serialized to JSON.
            """
        ),
    ] = True,
) -> Any:
    """
    Convert any object to something that can be encoded in JSON.

    This is used internally by FastAPI to make sure anything you return can be
    encoded as JSON before it is sent to the client.

    You can also use it yourself, for example to convert objects before saving them
    in a database that supports only JSON.

    Read more about it in the
    [FastAPI docs for JSON Compatible Encoder](https://fastapi.tiangolo.com/tutorial/encoder/).
    """
    custom_encoder = custom_encoder or {}
    if custom_encoder:
        if type(obj) in custom_encoder:
            return custom_encoder[type(obj)](obj)
        else:
            for encoder_type, encoder_instance in custom_encoder.items():
                if isinstance(obj, encoder_type):
                    return encoder_instance(obj)
    if include is not None and not isinstance(include, (set, dict)):
        include = set(include)  # type: ignore[assignment]  # ty: ignore[unused-ignore-comment]
    if exclude is not None and not isinstance(exclude, (set, dict)):
        exclude = set(exclude)  # type: ignore[assignment]  # ty: ignore[unused-ignore-comment]
    if isinstance(obj, BaseModel):
        obj_dict = obj.model_dump(
            mode="json",
            include=include,
            exclude=exclude,
            by_alias=by_alias,
            exclude_unset=exclude_unset,
            exclude_none=exclude_none,
            exclude_defaults=exclude_defaults,
        )
        return jsonable_encoder(
            obj_dict,
            exclude_none=exclude_none,
            exclude_defaults=exclude_defaults,
            sqlalchemy_safe=sqlalchemy_safe,
        )
    if dataclasses.is_dataclass(obj):
        assert not isinstance(obj, type)
        obj_dict = dataclasses.asdict(obj)
        return jsonable_encoder(
            obj_dict,
            include=include,
            exclude=exclude,
            by_alias=by_alias,
            exclude_unset=exclude_unset,
            exclude_defaults=exclude_defaults,
            exclude_none=exclude_none,
            custom_encoder=custom_encoder,
            sqlalchemy_safe=sqlalchemy_safe,
        )
    if isinstance(obj, Enum):
        return obj.value
    if isinstance(obj, PurePath):
        return str(obj)
```

```
    if isinstance(obj, (str, int, float, type(None))):
        return obj
    if isinstance(obj, PydanticUndefinedType):
        return None
    if isinstance(obj, dict):
        encoded_dict = {}
        allowed_keys = set(obj.keys())
        if include is not None:
            allowed_keys &= set(include)
        if exclude is not None:
            allowed_keys -= set(exclude)
        for key, value in obj.items():
            if (
                (
                    not sqlalchemy_safe
                    or (not isinstance(key, str))
                    or (not key.startswith("_sa"))
                )
                and (value is not None or not exclude_none)
                and key in allowed_keys
            ):
                encoded_key = jsonable_encoder(
                    key,
                    by_alias=by_alias,
                    exclude_unset=exclude_unset,
                    exclude_none=exclude_none,
                    custom_encoder=custom_encoder,
                    sqlalchemy_safe=sqlalchemy_safe,
                )
                encoded_value = jsonable_encoder(
                    value,
                    by_alias=by_alias,
                    exclude_unset=exclude_unset,
                    exclude_none=exclude_none,
                    custom_encoder=custom_encoder,
                    sqlalchemy_safe=sqlalchemy_safe,
                )
                encoded_dict[encoded_key] = encoded_value
        return encoded_dict
    if isinstance(obj, (list, set, frozenset, GeneratorType, tuple, deque)):
        encoded_list = []
        for item in obj:
            encoded_list.append(
                jsonable_encoder(
                    item,
                    include=include,
                    exclude=exclude,
                    by_alias=by_alias,
                    exclude_unset=exclude_unset,
                    exclude_defaults=exclude_defaults,
                    exclude_none=exclude_none,
                    custom_encoder=custom_encoder,
                    sqlalchemy_safe=sqlalchemy_safe,
                )
            )
        return encoded_list

    if type(obj) in ENCODERS_BY_TYPE:
        return ENCODERS_BY_TYPE[type(obj)](obj)
    for encoder, classes_tuple in encoders_by_class_tuples.items():
        if isinstance(obj, classes_tuple):
            return encoder(obj)
    if is_pydantic_v1_model_instance(obj):
        raise PydanticV1NotSupportedError(
            "pydantic.v1 models are no longer supported by FastAPI."
            f" Please update the model {obj!r}."
        )
    try:
        data = dict(obj)
    except Exception as e:
        errors: list[Exception] = []
        errors.append(e)
        try:
            data = vars(obj)
        except Exception as e:
            errors.append(e)
```

```
            raise ValueError(errors) from e
    return jsonable_encoder(
        data,
        include=include,
        exclude=exclude,
        by_alias=by_alias,
        exclude_unset=exclude_unset,
        exclude_defaults=exclude_defaults,
        exclude_none=exclude_none,
        custom_encoder=custom_encoder,
        sqlalchemy_safe=sqlalchemy_safe,
    )
```